

SkillVerify Project - Detailed Technical Overview

1. Executive Summary

SkillVerify is a GitHub-based hiring analysis platform. It ingests resumes (PDF/DOCX) or a bulk ZIP of resumes, extracts the candidate's GitHub profile, analyzes repositories and activity, compares resume skills to GitHub evidence, and generates a comprehensive report with scoring and recommendations. The frontend presents dashboards and can request a PDF rendered from the HTML report for professional output.

2. High-Level Architecture

- Frontend: React + Vite client for upload, visualization, and report export.
- Backend: Python Flask API orchestrating parsing, GitHub analysis, scoring, and report generation.
- External services: GitHub REST API.

Data Flow Summary:

1. User uploads resume(s).
2. Backend parses resume text and extracts GitHub URL.
3. Backend fetches GitHub profile and repositories.
4. Analysis services compute tech stack, skills match, code quality, and contributions.
5. Scoring engine computes overall rating.
6. Report generator builds JSON report.
7. Frontend renders report and optionally requests PDF rendering from HTML.

3. Core Features

- Single resume analysis (PDF/DOCX).
- Bulk resume analysis via ZIP.
- GitHub profile and repo insight extraction.
- Skill verification (match, missing, extra) with evidence and usage metrics.
- Code quality grading (README quality, docs signals, code organization, commit quality, optional code health).
- Contribution analysis (commit activity, streaks, top repos).
- Weighted scoring and recommendation.
- PDF generation from HTML using Playwright/Chromium for styling fidelity.

4. Tech Stack and Dependencies

Backend (Python)

- Flask: REST API, request handling, JSON responses.
- Flask-CORS: Cross-origin support for frontend.
- python-dotenv: Loads environment variables (GITHUB_TOKEN).
- requests: GitHub API calls.
- pdfplumber: Primary PDF text extraction.
- PyPDF2: PDF text extraction fallback.

- PyMuPDF (pymupdf): PDF link extraction.
- python-docx: DOCX parsing.
- reportlab: Structured PDF generator (fallback).
- playwright: HTML-to-PDF rendering via Chromium.

Frontend (React)

- React 18: Component UI.
- Vite: Build tooling and dev server.
- axios: API requests.
- recharts: Charts and visual analytics.
- react-table: Sorting and tabular data.
- lucide-react: Iconography.

5. Backend Components and Responsibilities

5.1 backend/app.py

Primary Flask application.

- Configuration: upload folder, max file size, allowed extensions.
- Routes:
 - /api/analysis/upload: single resume analysis.
 - /api/analysis/bulk-upload: ZIP bulk analysis.
 - /api/analysis/download-report: receives HTML and returns PDF.
 - /api/analysis/debug-parse: debug resume extraction.
 - /api/analysis/test-github/: GitHub fetch test endpoint.
- Orchestration: process_resume_file() runs the analysis pipeline.
- Error handlers: 413 (file too large), 500 (server error).
- Cleanup: uploaded file removal in finally block.

5.2 backend/services/resume_parser.py

Resume parsing and extraction.

- PDF parsing:
 - pdfplumber: preferred for complex PDFs.
 - PyPDF2: fallback.
- DOCX parsing:
 - python-docx for paragraph extraction.
- GitHub extraction:
 - PyMuPDF link extractor for embedded URL detection.
 - Regex fallback on raw text.
- Field extraction:
 - Name: heuristics from first lines.
 - Email: regex.
 - Phone: regex.
 - Skills: keyword matching + normalization map.

5.3 backend/services/pdf_link_extractor.py

Dedicated GitHub URL extraction from PDFs.

- Uses PyMuPDF to parse links and text.
- Filters out non-profile GitHub URLs.
- Returns the best candidate username.

5.4 backend/services/github_service.py

GitHub API client and data normalization.

- Fetches:
 - Profile data.
 - Repository list.
 - Repo contents and file content.
 - Languages, commits, contributors.
- Handles API errors:
 - 404 -> user not found.
 - 403 -> rate limit exceeded.
- Aggregates language stats and contribution activity.

5.5 backend/services/tech_stack_analyzer.py

Detects technologies used in repos.

- Uses file patterns, config files, package names, file extensions, and import heuristics.
- Samples top repositories by star and recency.
- Builds detected_skills with confidence, evidence, usage frequency, and repositories.

5.6 backend/services/skill_matcher.py

Matches resume skills to GitHub evidence.

- Normalizes skill names and variants.
- Exact matching and fuzzy matching.
- Produces:
 - matched_skills (with confidence and evidence).
 - missing_skills (resume claims not found).
 - extra_skills (GitHub skills not in resume).
- Computes statistics for scoring and authenticity.

5.7 backend/services/code_quality_analyzer.py

Assesses repo quality via static heuristics.

- Signals:
 - README presence and quality (structure + optional readability).
 - Root-level docs / examples / architecture doc signals.
 - Code organization from top-level folder patterns.

- Commit frequency + commit message quality (with anti-repetition penalty when available).
- Optional code health (real source analysis via Radon/Esprima/Javalang).
- Produces per-repo and overall grades.
- Generates strengths, weaknesses, suggestions.

5.8 backend/services/contribution_analyzer.py

Analyzes commit activity and contributions.

- Separates owned vs forked repos.
- Computes commit counts, top repos, activity patterns.
- Calculates streak information (consistent activity).

5.9 backend/services/scoring_engine.py

Computes overall score and rating.

- Weighted scoring with defined weights.
- Produces breakdown and rating label.

5.10 backend/services/report_generator.py

Builds final report and PDF.

- Generates structured JSON report from analysis outputs.
- PDF generation:
 - Primary: Playwright/Chromium for HTML-to-PDF.
 - Fallback: ReportLab structured PDF generator.

6. Frontend Components and Responsibilities

6.1 frontend/src/App.jsx

Top-level application state.

- Manages report, bulk results, loading, and error states.
- Switches views between Upload, BulkDashboard, and Dashboard.

6.2 frontend/src/components/UploadSection.jsx

Resume upload UI.

- Drag-and-drop file selection.
- Single resume vs bulk ZIP mode.
- Client-side validation for file size and type.
- Calls /api/analysis/upload or /api/analysis/bulk-upload.

6.3 frontend/src/components/BulkDashboard.jsx

Bulk summary UI.

- Table of candidate summaries with sorting and filtering.

- CSV export.
- Allows selecting an individual report for detailed view.

6.4 frontend/src/components/Dashboard.jsx

Single report UI and PDF export.

- Renders candidate summary, charts, and analysis sections.
- Builds a detailed HTML report template.
- Sends HTML to `/api/analysis/download-report` to receive PDF.

7. End-to-End Flow (Detailed)

7.1 Single Resume Upload

1. User selects a PDF/DOCX resume.
2. Frontend posts FormData to `/api/analysis/upload`.
3. Backend validates file type and size.
4. ResumeParser extracts text and GitHub username.
5. GitHubService fetches profile and repositories.
6. TechStackAnalyzer scans repos for technologies.
7. SkillMatcher matches resume skills vs GitHub.
8. CodeQualityAnalyzer evaluates repo quality.
9. ContributionAnalyzer computes commit activity.
10. ScoringEngine computes overall and breakdown scores.
11. ReportGenerator composes report JSON.
12. Report is returned to frontend for visualization.

7.2 Bulk Upload (ZIP)

1. User uploads ZIP of resumes.
2. Backend extracts ZIP to a temp folder.
3. Each file is processed via `process_resume_file()`.
4. Summary rows are computed per candidate.
5. Aggregates are computed (average score and counts).
6. Summary + report array returned to frontend.

7.3 PDF Generation

1. Frontend builds HTML report template.
2. Sends HTML to `/api/analysis/download-report`.
3. ReportGenerator uses Playwright to render HTML in Chromium.
4. PDF is returned to frontend for download.

8. Scoring Formulas and Logic

8.1 Overall Score (Weighted)

- $\text{overall} = \text{skill_authenticity}0.60 + \text{code_quality}0.10 + \text{commit_activity}0.10 + \text{tech_stack}0.10 + \text{profile_signal}0.10$

8.2 Commit Activity Score

- $\text{recent_score} = \min(100, \text{recent_commits} * 4)$
- $\text{volume_score} = \min(100, (\text{total_commits} / 200) * 100)$
- $\text{commit_activity} = \text{recent_score} * 0.6 + \text{volume_score} * 0.4$

8.3 Tech Stack Score

- $\text{breadth} = \min(\text{detected_skills}, 25) / 25 * 100$
- $\text{avg_confidence} = \text{average}(\text{skill.confidence})$
- $\text{tech_stack} = \text{breadth} * 0.6 + \text{avg_confidence} * 0.4$

8.4 Profile Signal Score

Profile scoring intentionally avoids social popularity signals (followers).

The profile signal blends:

- Profile completeness (bio/name/location/company/blog)
- Portfolio depth (public repos, saturates at 25)
- Active volume (active repos, saturates at 8)
- Activity consistency (active repos / total repos)
- Account maturity (saturates at ~4 years)

Formula:

- $\text{profile_signal} = \text{completeness} * 0.20 + \text{depth} * 0.25 + \text{active_volume} * 0.30 + \text{consistency} * 0.15 + \text{maturity} * 0.10$

For full details, see [docs/scoring-deep-dive.md](#).

9. Exception Handling

9.1 API-Level Errors

- Missing resume file -> 400 error.
- Unsupported file type -> 400 error.
- Missing GitHub URL -> 400 error with debug info.
- File too large -> 413 error handler.
- Unhandled exceptions -> 500 error handler.

9.2 Service-Level Resilience

- Resume parsing uses multiple fallbacks (pdfplumber -> PyPDF2).
- GitHub link extraction uses PyMuPDF link parsing with regex fallback.
- Repository analysis exceptions are caught per repo to allow progress.
- GitHub API errors are surfaced with meaningful messages.

9.3 PDF Rendering Errors

- Playwright errors bubble to API and return 500.
- ReportLab generator exists as fallback.

10. Data Schema (Report Output)

Key report sections:

- metadata: generatedAt, reportVersion, analysisType
- candidateSummary: name, email, githubUsername, githubUrl, bio, etc.
- githubProfile: stats, activity, languageDistribution
- technicalAnalysis: detectedSkills, categories, totalDetected
- skillMatchingReport: summary, matchedSkills, missingSkills, extraSkills
- codeQualityReport: overallScore, grade, metrics, per-repo breakdown
- contributionReport: summary, topRepositories, activity, streak
- scoringAnalysis: overallScore, rating, breakdown
- recommendation: decision, confidence, reasoning

11. Configuration and Environment

- Environment variables:
 - GITHUB_TOKEN (optional, improves rate limit and access)
 - FLASK_ENV (development toggles error detail)
 - CLIENT_URL (CORS origin override)
- Upload folder: /uploads
- Max upload size: 5 MB per file (single); 20 MB ZIP in frontend validation.

12. Observed Limitations and Notes

- GitHub API rate limits apply without token.
- Resume skill extraction is heuristic and may miss context.
- Bulk ZIP path traversal risk is mitigated by extracting to temp and filtering.
- Some legacy Node dependencies exist in root package.json, but Flask is the active backend.

13. Suggested Enhancements (Optional)

- Add persistent database storage for reports and candidate history.
- Add caching for GitHub API results to reduce requests.
- Add async background jobs for bulk processing.
- Add user authentication for multi-tenant usage.
- Expand skill ontology with domain-specific vocabularies.